Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Отчёт о нагрузочных испытаниях

Версия 1.0.0: организация на 1000–2000 сотрудников · версия 1.0.0

30 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Итог 4

3 Стенд и метод 5

4 Сайзинг (E51-02) 6

4.1 Запас 6

5 Узкие места (E51-06) 8

5.1 Ошибки продукта, найденные прогонами 9

6 Пики (E51-03) 12

7 Длительная работа (E51-04) 13

8 Отказы под нагрузкой (E51-07) 14

9 Рост данных (E51-08) 15

10 Воспроизведение 16

1 О документе

Отчёт о нагрузочных испытаниях «Планировщика отпусков» версии 1.0.0 для организации на 1000–2000 сотрудников (эпик E51): сайзинг, узкие места с доказательствами и исправлениями, пики, длительная работа, отказы под нагрузкой и рост данных. Модель нагрузки и профили — `docs/performance/workday-model.md`, порядок прогона и сбора доказательств — `docs/performance/runbook.md`, диагностика — `docs/runbooks/diagnostics.md`.

Каждая цифра отчёта воспроизводится командой из раздела «Воспроизведение»; итоговые файлы генератора лежат в `docs/performance/results/`.

2 Итог

ИСТОРИЯ	ТРЕБОВАНИЕ	РЕЗУЛЬТАТ
E51-02 Сайзинг	`average` и `spike` в p95 чтения ≤ 300 мс, записи ≤ 500 мс, без ошибок	**1 vCPU** (API и PostgreSQL на одном ядре) обслуживает **2000 сотрудников**: `average` — p95 чтения 48 мс, записи 68 мс; `spike` (вход всей организации за 15 минут) — 74 и 145 мс, вход p95 625 мс, 0 ошибок. Запас: на 1 vCPU — ×2 от часа пик, на рекомендуемых 2 vCPU — ×8
E51-06 Узкие места	Только по доказательствам, исправлены и перепроверены, прогон с порогами в CI	Найдено 5 узких мест и 4 ошибки продукта, 3 недостатка генератора; исправлено всё, кроме намеренно дорогой проверки пароля; каждое исправление перепроверено; две гипотезы не подтвердились. Задание CI «Нагрузочный дымовой прогон с порогами»
E51-03 Пики	Без ошибок и в требованиях; очередь писем разбирается в срок	После исправления волны уведомлений: фон p95 чтения 94 мс, записи 84 мс, 0 ошибок; 2000 приглашений доставлены за 45,6 с; выгрузки T-7 и графика — до 3,7 с
E51-04 Длительная работа	8 часов: задержки и память не растут	7 ч 15 мин без ошибок: прогон прерван перезапуском стенда испытаний, первый — на 5 ч 16 мин. Память не растёт (куча V8 204–207 МБ), потоки и соединения стабильны, плановые задачи без ошибок; p95 чтения 35 → 49 мс — отменённые заявки копятся у работающих генератора, а не продукта
E51-07 Отказы	Ошибки только в окне отказа, клиенты восстанавливаются сами	Ошибки только в окнах отказов: процесс API — 0, перезапуск PostgreSQL — 12 ответов `503` за < 10 с, перезапуск API — 20 с; все потоки событий и сеансы восстановились сами. Исправлены ошибка 8 (остановка API ждала потоки событий 15 с; перезапуск API — 18 → 3 с) и ошибка 9 (`500` вместо `503` при перезапуске PostgreSQL)
E51-08 Рост данных	Размеры, планы запросов, сроки хранения, копирование	5 лет на 2000 сотрудников — 67 МБ со сроками хранения (125 МБ без них); документы растут примерно на 4,5 МБ в год; копия — 1,2 с, восстановление — 1,9 с; полный просмотр большой таблицы — только у графика организации, 3 мс; архив WAL — до 0,5 ГБ за рабочий день

3 Стенд и метод

ПАРАМЕТР	ЗНАЧЕНИЕ
Узел	4 vCPU Intel Xeon 2,1 ГГц, 15 ГБ ОЗУ, Linux
Ограничение ресурсов	API и PostgreSQL — на одном ядре (`taskset -c 0`) или двух (`0,1`); генератор и сборщик ресурсов — на двух других (`tools/load/pinned.sh`)
API	Node.js 22, сборка версии 1.0.0; процессов — по числу ядер стенда (`WEB_CONCURRENCY=auto`): 1 на 1 vCPU
PostgreSQL	16.15 на стенде нагрузки (в поставке — 17: планировщик и исполнитель запросов для этих запросов те же), `shared_buffers` 128 МБ, `pg_stat_statements`
Данные	`load-seed.ts` : 1000 и 2000 сотрудников с историей за 5 лет — 69 и 127 МБ (`workday-model.md` , «Данные стенда»); наполнение заново перед каждым прогоном
Генератор	`tools/load/run.mjs` : роли сотрудника, согласующего и кадровой службы, вкладки с потоком событий, утренний приход; поведение клиента повторяется, включая обновление токена, переподключение потока событий и случайную задержку перечитывания графика
Доказательства	Итог генератора и лента по минутам, ресурсы каждые 5 с (`sample.mjs` : ядра API, PostgreSQL и генератора, RSS и куча V8, соединения), `pg_stat_statements` , журнал медленных запросов API, профиль процессора

Генератор в прогонах не упирался в свои ядра: средняя загрузка — 0,05–0,2 ядра.

4 Сайзинг (E51-02)

ЧИСЛЕННОСТЬ	ЯДЕР СТЕНДА	ПРОФИЛЬ	ЗАПРОСОВ В СЕКУНДУ	P95 ЧТЕНИЯ, МС	P95 ЗАПИСИ, МС	ВХОД P95, МС	ОШИБОК	ЯДЕР API: СРЕДНЕЕ / ПИК	RSS API, МБ
2000	1	`average`	31	**48**	**68**	540	0*	0,25 / 0,86	679
2000	1	`spike`	62	**74**	**145**	625	0	0,48 / 0,90	990
1000	1	`average`	16	**33**	**47**	152	0	0,13 / 0,35	558
1000	1	`spike`	31	**40**	**66**	149	0	0,26 / 0,56	604

* Два ответа `401` на `GET /auth/sessions` — недостаток генератора (не обновлял токен на путях `/auth/…`), исправлен; сервер отвечал верно.

PostgreSQL в этих прогонах занимает 0,05–0,15 ядра: на 2000 сотрудников ядро нагружает API, а не БД. Пик загрузки ядра (0,9) — утренний приход всей организации: проверка пароля scrypt на каждом входе. Отсюда рекомендации:

ЧИСЛЕННОСТЬ	МИНИМАЛЬНАЯ КОНФИГУРАЦИЯ	РЕКОМЕНДУЕМАЯ	ПОЧЕМУ
до 1000	1 vCPU, 2 ГБ ОЗУ, 20 ГБ диска	2 vCPU, 4 ГБ, 40 ГБ	Запас на выгрузки, резервное копирование и обновление без простоя
до 2000	1 vCPU, 2 ГБ ОЗУ, 20 ГБ диска	2 vCPU, 4 ГБ, 40 ГБ	На 1 vCPU утренний приход занимает ядро на 90 %, а запас — двукратный; на 2 vCPU запас восьмикратный, процессам API на нём нужно до 2,1 ГБ (раздел «Запас»)

Число ядер проверено прогонами с привязкой процессов; ОЗУ в прогонах не ограничивалась, а измерялась: процесс API с 715–776 открытыми потоками событий — до 1 ГБ RSS при куче V8 около 200 МБ после сборки мусора; PostgreSQL — `shared_buffers` 128 МБ и по 10–20 МБ на соединение. Отсюда минимум 2 ГБ на узел и 4 ГБ с запасом. Диск: база 2000 сотрудников за 5 лет — 127 МБ, копии `pg_dump` за 14 дней и архив WAL — раздел «Рост данных». Запас по ядрам — раздел «Запас» ниже.

4.1 Запас

Профиль `stress` на 2000 сотрудников и 1 vCPU: ступени ×1, ×2, ×4, ×8 и ×16 от часа пик перед сроком кампании, по 5 минут; 600 вкладок без действий держат поток событий всё время прогона.

СТУПЕНЬ	РАБОТАЮТ	ЗАПРОСОВ В СЕКУНДУ	P95 ЧТЕНИЯ, МС	P95 ЗАПИСИ, МС	ОШИБОК	ЯДРА: API + POSTGRESQL
×1	115	41	664*	805*	0	0,22 + 0,05
×2	230	57	**76**	**120**	0	0,40 + 0,09
×4	460	108	890	1537	0	0,74 + 0,17
×8	920	112	9485	12 921	522 (1,5 %)	0,83 + 0,15

СТУПЕНЬ	РАБОТАЮТ	ЗАПРОСОВ В СЕКУНДУ	P95 ЧТЕНИЯ, МС	P95 ЗАПИСИ, МС	ОШИБОК	ЯДРА: API + POSTGRESQL
×16	1840	157	23 043	19 647	9223 (19,6 %)	0,87 + 0,11

* Ступень ×1 включает разгон: 715 вкладок входят за первую минуту, и проверка пароля scrypt занимает ядро на 90 %. В минуты 0–1 p95 чтения — 2,6–3,0 с, в минуты 2–4 — 44–49 мс, как в `average`. Ядра указаны без минуты разгона.

Тот же профиль на рекомендуемой конфигурации — 2 vCPU: два процесса API (`WEB_CONCURRENCY=auto`) и PostgreSQL на тех же двух ядрах.

СТУПЕНЬ	РАБОТАЮТ	ЗАПРОСОВ В СЕКУНДУ	P95 ЧТЕНИЯ, МС	P95 ЗАПИСИ, МС	ОШИБОК	ЯДРА: API + POSTGRESQL	RSS ПРОЦЕССОВ API, МБ
×1	115	43	**24**	**43**	0	0,24 + 0,05	906
×2	230	57	**27**	**46**	0	0,39 + 0,09	1484
×4	460	111	**40**	**67**	0	0,77 + 0,18	1897
×8	920	218	**166**	**427**	0	1,32 + 0,34	2092
×16	1840	284	6517	9434	9 (`503`)	1,62 + 0,35	2498

RSS — сумма основного процесса и двух рабочих; на ×8 открыто 1520 потоков событий, на ×16 — 2440.

Выводы:

- **Запас на 1 vCPU — двукратный.** При вдвое большей активности, чем в последний час кампании, p95 чтения — 76 мс, ядро занято наполовину. На ×4 ядро занято полностью (0,9): ошибок нет, но запросы ждут в очереди, p95 — 0,5–1,7 с по минутам.
- **Первым кончается процессор, а не база и не память.** PostgreSQL не превышает 0,17 ядра, RSS процесса API — не больше 772 МБ.
- **Второе ядро даёт восьмикратный запас.** На 2 vCPU ×8 от часа пик укладывается в требования (p95 чтения 166 мс, записи 427 мс, без ошибок), а разгон 715 вкладок за минуту не заметен (p95 чтения в первую минуту — 29 мс). Запас растёт больше чем вдвое: на одном ядре API, PostgreSQL, сборка мусора и проверка паролей отнимают время друг у друга, на двух — работают параллельно. Отсюда рекомендация 2 vCPU и 4 ГБ для 2000 сотрудников: на ×8 процессам API нужно 2,1 ГБ.
- **За пределом запаса сервер не падает.** На ×8 и ×16 операции с транзакцией не дожидаются соединения с БД за 2 с и отвечают `503` с `Retry-After` (4715 ответов, все — Prisma `P2028`); остальные запросы ждут в очереди до 20–80 с, часть соединений генератор не может открыть за 10 с. Процесс API работает весь прогон без перезапусков, RSS не растёт.
- **Перегрузка нашла две ошибки продукта** (раздел «Ошибки продукта, найденные прогонами»): ложное завершение сеанса новой вкладки и выход на экран входа при `503` на обновлении токена. Обе исправлены. Повторный прогон `stress` с исправлением (`results/2000-1cpu-stress-fixed`) дал те же ступени (×2 — p95 чтения 76 мс, ×4 — 930 мс) и ни одного события `revoked`; 9 сбоев потока событий на ×8–×16 — билет или поток, которые генератор не смог открыть за 10 с.

5 Узкие места (E51-06)

Узкое место признаётся только при трёх доказательствах: симптом в итоге генератора, совпадающая метрика или профиль, воспроизведение отдельной командой (`runbook.md`, раздел 7). Ранжирование — по влиянию на требования на 1 vCPU.

№	УЗКОЕ МЕСТО	СИМПТОМ	ПРИЧИНА (ДОКАЗАТЕЛЬСТВО)	ИСПРАВЛЕНИЕ	ПЕРЕПРОВЕРКА
1	Волна перечитываний графика после изменения данных	`average` на 2000, 1 vCPU: p95 чтения **435 мс** при требовании 300; у `GET /schedule` p50 19 мс, а p95 637 мс	531 из ~620 медленных запросов — `/schedule`; событие `data` приходит во все 715 вкладок сразу, видимые перечитывают график в одну секунду; одинаковые запросы отдела промахивались мимо кэша и считались параллельно (`ScheduleView.vue`, `schedule.controller.ts`)	Клиент перечитывает через случайные 0–10 с; одинаковые запросы «в полёте» ждут один расчёт (`f709e9d`)	Тот же прогон: p95 чтения **48 мс**, у графика 637 → **30 мс**; медленных запросов графика 531 → 1
2	Выгрузка формы Т-7 останавливала процесс API	В `average` на 1 vCPU выгрузка шла 6,5 с; все запросы, пришедшие в это время, ждали 0,5–2,1 с	Профиль процессора: ExcelJS 2,2 с и сжатие zip 0,5 с на четыре выгрузки — в основном потоке; пробы `/healthz` во время выгрузки — до 366 мс	Книга собирается в `worker_threads` (`2d2c966`)	Пробы `/healthz` во время выгрузки — не дольше 9 мс; файл тот же
3	Очередь писем: 1,3 письма в секунду на процесс	Замер на сборке до исправления (`743c7a9^`): ровно 20 писем каждые 15 с — 320 из 2000 приглашений за 4 минуты (`results/mail-before`)	`MailService`: одна пачка 20 писем за проход, проход раз в 15 с; 2000 приглашений кампании на одном процессе — около 25 минут при требовании 15	Проход разбирает очередь пачками до конца (до 10 с), SMTP с пулом соединений (`743c7a9`)	Пики: 2000 приглашений разобраны за 45,6 с, около 45 писем в секунду
4	Вход занимает ядро при утреннем приходе	`spike` на 1 vCPU: вход p95 625 мс, p99 1,1 с; ядро API — до 0,9	scrypt (N = 16384) — намеренно дорогая проверка пароля (ADR-003), около 50 мс процессора на вход; 2000 входов за 15 минут	Не исправляется: стойкость хеша важнее; в требованиях (вход ≤ 1 с p95). Для организаций с волной входов — 2 vCPU (сайзинг) или единый вход OIDC, где пароль не проверяется	—

№	УЗКОЕ МЕСТО	СИМПТОМ	ПРИЧИНА (ДОКАЗАТЕЛЬСТВО)	ИСПРАВЛЕНИЕ	ПЕРЕПРОВЕРКА
5	Волна запросов уведомлений после уведомления всей организации	Пики на 2000 сотрудников и 1 vCPU: в минуты открытия кампании и напоминания p95 чтения 2,2 и 9,8 с; у `GET /notifications` p50 1,2 с; 3 ответа `503` на подачу заявки	Журнал медленных запросов: за 10 с после открытия кампании — 735 медленных запросов, из них 657 — `/notifications`; после напоминания — 554 и 391. Событие `notifications` приходит во все 715 вкладок сразу, и каждая перечитывает список; клиент перечитывал ещё и счётчик заявок, генератор этого не повторял — настоящая волна вдвое больше	Значок берёт число непрочитанных из события; список читается при открытии панели; счётчик заявок перечитывают только согласующие, через 0–10 с (`App.vue` , `stores/notifications.ts`)	Тот же прогон пиков: p95 чтения фона 450 → **94 мс**, ошибок 3 → 0; медленных запросов `/notifications` за прогон 1131 → 3; T-7 8,2–9,0 → 3,0–3,7 с

5.1 Ошибки продукта, найденные прогонами

Не узкие места, а ошибки: под нагрузкой они проявлялись чаще, чем в тестах, но возможны и при обычной работе.

№	ОШИБКА	СИМПТОМ	ПРИЧИНА (ДОКАЗАТЕЛЬСТВО)	ИСПРАВЛЕНИЕ	ПЕРЕПРОВЕРКА
6	Новая вкладка получала ложный `revoked` , и клиент выводил сотрудника на экран входа	`spike` на 2000 сотрудников: 1 событие `revoked` ; `stress` : 199. Генератор не выходит, не меняет пароль и не завершает сеансы	199 событий = 199 переподключений, а `hello` лишь на 7 больше числа вкладок: отзывались потоки, ещё не получившие `hello` . Проверка сеансов (`EventsService.closeRevoked`) снимала список сеансов, ждала два запроса к БД и перебирала уже текущие потоки — подключившийся за это время считался завершённым	Проверяются только потоки из снимка до запросов (`00a1ec1`); `events.unit.test.ts` воспроизводит гонку без БД	Тот же `stress` : `revoked` 199 → **0**
7	`503` при обновлении токена выводил сотрудника на экран входа	`stress` ×8 и ×16: 2640 из 2993 обновлений токена — `503` (транзакция обновления не дождалась соединения с БД за 2 с)	`web/src/api/client.ts` считал любой неуспех обновления потерей сеанса; то же случилось бы при перезапуске PostgreSQL у всех, чей access-токен истёк в окне отказа	На экран входа ведёт только отказ (`401`); при `503` и обрыве — сообщение «повторите» с кодом запроса, сеанс сохраняется (`a3a49ec` , `client.spec.ts`)	Модульные тесты клиента; два из пяти падают на прежнем коде

№	ОШИБКА	СИМПТОМ	ПРИЧИНА (ДОКАЗАТЕЛЬСТВО)	ИСПРАВЛЕНИЕ	ПЕРЕПРОВЕРКА
8	Остановка API ждала открытые потоки событий до таймаута	Отказы под нагрузкой: от `SIGTERM` до запуска нового API — 16 с, и всё это время `ECONNREFUSED` (526 ошибок)	Журнал: процессы получили `SIGTERM` в 15:50:09, новый API запущен в 15:50:25 — по таймауту основного процесса 15 с. Nest закрывает HTTP-сервер между `beforeApplicationShutdown` и `onApplicationShutdown`; сервер ждёт открытые соединения, а потоки закрывались в `onApplicationShutdown`. В Docker — те же 10 с до `SIGKILL` при каждом обновлении	Потоки закрываются в `beforeApplicationShutdown`; `events.e2e`: остановка с открытым потоком — до исправления тест висит до таймаута 30 с, после — весь файл 1,1 с	Повторный прогон отказов: от `SIGTERM` до ответа нового API 18 → **3 с**, ошибок перезапуска 526 → 68
9	Перезапуск PostgreSQL давал `500` вместо `503` на сырых запросах	Повторный прогон отказов: 2 ответа `500` — `GET /schedule` и `GET /analytics/overview` — в секунду перезапуска PostgreSQL	Журнал сервера: `unhandled error`, Prisma `P2010` «Raw query failed. Code: `57P01`» — соединение закрыто администратором. Запросы через `$queryRaw` сообщают о закрытом соединении этим кодом, а фильтр ошибок знал только `P1001`, `P1017` и подобные	Коды недоступности PostgreSQL в `P2010` (`57P01` – `57P03`, `08xxx`, `53300`) — `503` с `Retry-After`; `db-unavailable.unit.test.ts` — два из трёх случаев падают на прежнем коде	Итоговый прогон отказов: в перезапуск PostgreSQL 10 ответов `503`, `500` — ни одного; сырой запрос с `57P01` — `503`

Недостатки генератора, найденные по ходу (без них цифры были бы неверны):

НЕДОСТАТОК	КАК ПРОЯВЛЯЛСЯ	ИСПРАВЛЕНИЕ
Не обновлял токен по `401` на путях `/auth/…`	2–7 ошибок `401` на `GET /auth/sessions` за прогон	Как в `web/src/api/client.ts`: повтор после обновления на всех путях, кроме входа и обновления
Не переподключал поток событий	После перезапуска API «вкладки исчезали», отказы нельзя было измерить	Переподключение через 5 с, как `EventSource` браузера
Перечитывал график и список уведомлений по событию, как клиент до 1.0.0	Модель не совпала бы с клиентом после исправлений 1 и 7	Повторяет `ScheduleView` и `App.vue`; `--no-jitter` — прежнее поведение для сравнения
Писал итог только в конце прогона	8-часовой прогон, прерванный перезапуском стенда на шестом часе, потерял ленту задержек; ресурсы `sample.mjs` пишет построчно, они сохранились	Итог пишется на каждом шаге ленты, не реже раза в минуту, через временный файл; `complete: false` — промежуточный

Гипотезы, которые не подтвердились:

ГИПОТЕЗА	ПРОВЕРКА	ВЫВОД
Утечка памяти на потоках событий	RSS API рос 227 → 675 МБ за 7 минут; куча V8 по `/metrics` — пила 180 → 430 → 181 МБ, после сборки мусора возвращается к ~190 МБ	Не утечка: ленивая сборка мусора; рабочий объём ~200 МБ кучи на 715 потоков. Длительная работа — раздел «Длительная работа»

ГИПОТЕЗА	ПРОВЕРКА	ВЫВОД
История за 5 лет замедляет график организации	Запрос графика организации — полный просмотр `vacation_requests` (35 тыс. строк); `EXPLAIN (ANALYZE, BUFFERS)`: просмотр 3 мс из 25, остальное — форматирование 5 тыс. строк результата	Не узкое место: 10 лет истории добавят ~3 мс; ответ кэшируется по версии данных

Прогон с порогами в CI: задание «Нагрузочный дымовой прогон с порогами» на каждый push — 300 сотрудников с историей за год, профиль `smoke`, `run.mjs --slo` завершается с ошибкой при p95 чтения > 300 мс, записи > 500 мс, ошибках или сбоях потока событий.

6 Пики (E51-03)

Пики, которые создаёт сервер по действию кадровой службы или по расписанию (`peaks.mjs`), — на фоне `average` на 2000 сотрудников и 1 vCPU: 25 минут, 115 работающих и 600 вкладок без действий. Почта уходит на SMTP-приёмник (`smtp-sink.mjs`), очередь писем опрашивается раз в 15 с. Утренний вход всей организации за 15 минут — профиль `spike`, раздел «Сайзинг».

Прогон выполнен дважды: до и после исправления волны запросов уведомлений (узкое место 5). Исправления 1–3 действуют в обоих прогонах.

ПИК	ТРЕБОВАНИЕ	ДО ИСПРАВЛЕНИЯ 5	ПОСЛЕ
Открытие кампании: приглашение 2000 сотрудникам, уведомления и письма одной транзакцией	Без ошибок	878 мс	741 мс
Разбор очереди: 2000 писем-приглашений	≤ 15 мин	101,6 с	**45,6 с**
Напоминание незапланировавшим (около 1900 писем)	Без ошибок	2278 мс	541 мс
Форма Т-7 на 2000 сотрудников, три выгрузки подряд	≤ 5 с	8,2–9,0 с	**3,0–3,7 с**
График за год в XLSX, три выгрузки подряд	≤ 5 с	1,0–2,5 с	1,0–3,7 с
Извещения: 400 отпусков с одного дня	До начала дня отпуска	39,6 с от появления отпусков	43,5 с
Утверждение графика: снимок 2000 сотрудников, уведомление всем	Без ошибок	980 мс	3429 мс
Фон `average`: p95 чтения / записи, ошибок	≤ 300 / ≤ 500 мс, 0	450 / 187 мс, 3 × `503`	**94 / 84 мс, 0**

Время извещений определяется опросом плановых задач (`JOBS_POLL_SECONDS`, 60 с): задача проходит все 400 отпусков за один проход. Утверждение графика измерено по одному разу в каждом прогоне (980 мс и 3,4 с); причину разницы одно измерение не показывает, ошибок нет в обоих.

Минуты действий кадровой службы на 1 vCPU. Пока идут три выгрузки Т-7 подряд и утверждение графика, p95 чтения по минутам — 387, 1103 и 551 мс (до исправления — 2164, 542 и 9820 мс). Книга Т-7 собирается в отдельном потоке и не останавливает процесс, но на одном ядре этот поток делит процессор с запросами. RSS процесса API в эти минуты — до 1,1 ГБ: куча V8 до 500 МБ, после сборки мусора — 240 МБ. Требование к прогону в целом выполнено. Организациям, где кадровая служба выгружает форму Т-7 в рабочие часы, рекомендуется 2 vCPU (раздел «Сайзинг»).

7 Длительная работа (E51-04)

Профиль `soak` на 2000 сотрудников и 1 vCPU — расчётный час кампании без перерыва: 115 работающих и 600 вкладок без действий. Требование — 8 часов. Оба прогона прервал перезапуск контейнера стенда испытаний — это свойство среды, а не программы: первый — на 5 ч 16 мин, второй — на 7 ч 15 мин. После первого перерыва генератор пишет итог по ходу прогона, поэтому от второго сохранились и задержки, и ресурсы. Полные 8 часов воспроизводятся на стенде организации командой из раздела «Воспроизведение».

Второй прогон (`results/2000-1cpu-soak`) по часам:

ЧАС	ЗАПРОСОВ В СЕКУНДУ	P95 ЧТЕНИЯ, МС (НАИБОЛЬШИЙ ЗА 5 МИН)	P95 ЗАПИСИ, МС	ОШИБОК	ЯДРА API / POSTGRESQL	RSS API, МБ	КУЧА V8, МБ
1	31	35 (39)	49	0	0,20 / 0,05	764	217
2	31	35 (39)	47	0	0,19 / 0,05	647	204
3	31	39 (43)	51	0	0,20 / 0,05	654	204
4	31	40 (46)	49	0	0,20 / 0,05	693	205
5	31	44 (51)	58	0	0,21 / 0,05	664	207
6	31	43 (46)	53	0	0,21 / 0,05	659	207
7	31	49 (55)	57	0	0,22 / 0,05	662	205
7:00–7:15	31	42 (47)	47	0	0,20 / 0,05	670	205

Итог: 800 482 запроса без ошибок, p95 чтения 41 мс, записи 52 мс; 715 потоков событий открыты всё время, ни одного сбоя и переподключения; 1,16 млн событий `data` и 8637 событий `notifications` .

- **Память не растёт.** Куча V8 со второго часа до конца — 204–207 МБ, RSS — 650–690 МБ. Первый прогон за 5 ч 16 мин показал то же: куча 204–207 МБ, RSS 641–735 МБ (`results/2000-1cpu-soak-interrupted`).
- **Соединения стабильны.** К PostgreSQL — 11 соединений всё время прогона (пул), потоков событий — 715.
- **Плановые задачи работают.** Проход задач — раз в 60 с, около 435 проходов: ни одной ошибки в журнале сервера. Суточные задачи (сроки хранения, журнал входов) выполнены по разу — `job_runs` .
- **Автоочистка справляется.** К пятому часу она прошла по `data_version` 289 раз, по `notifications` — 8, по `vacation_requests` , `audit_events` и `request_events` — по 1–2 раза. Мёртвых строк в любой таблице — не больше 10 тыс.
- **Медленных запросов — 99 за 7 ч 15 мин**, из них 80 — выгрузки Т-7 и графика.
- **p95 чтения медленно растёт: 35 → 49 мс** за 7 часов. Причина — свойство модели, а не продукта. 115 работающих все 7 часов — одни и те же сотрудники, и действие «подать и отменить заявку» оставляет у каждого по отменённой заявке примерно раз в 1,7 минуты. Через 5 часов у сотрудника `load.u00246` было 383 заявки, 365 из них отменены; ответ «Мои заявки» — 89 КБ, медиана 21,6 мс. У сотрудника с 16 заявками — 7 КБ и 9,4 мс; баланс и «Моё планирование» у обоих одинаковы, 6–8 мс (`results/2000-1cpu-soak/heavy-vs-light.txt`). У настоящего сотрудника — несколько заявок в год. Даже на седьмом часу p95 чтения вшестеро ниже требования.

8 Отказы под нагрузкой (E51-07)

`failures.sh` на 2000 сотрудников: профиль `average` 18 минут на 2 vCPU (два процесса API в кластере, PostgreSQL на тех же ядрах), лента генератора по 10 с. Три отказа по одному с паузой 4 минуты. Переключение на реплику под нагрузкой здесь не проверялось — для него нужен Docker. Его проверяет задание CI «Реплика PostgreSQL, архив WAL и переключение» (без нагрузки), порядок действий — `docs/runbooks/failover.md`.

ОТКАЗ	ЧТО ВИДЕЛИ КЛИЕНТЫ	ОКНО ОШИБОК	ВОССТАНОВЛЕНИЕ
Аварийное завершение одного из двух процессов API (`kill -9`)	Запросы шли без ошибок; потоки событий убитого процесса оборвались и переподключились	0 ошибок	Кластер запустил новый процесс за 1,3 с
Перезапуск API целиком: `SIGTERM`, затем запуск	526 ответов `ECONNREFUSED`, все 715 потоков событий переподключились	20 с — две ленты по 10 с	Новый API отвечает через 1,3 с после запуска; первая лента после отказа — 1818 запросов (отложенные действия), p95 чтения 172 мс, дальше — как до отказа. 16 с из 20 — ожидание остановки старого API: ошибка 8
Перезапуск PostgreSQL	12 ответов `503` (`P1017`, соединение закрыто сервером)	Меньше 10 с — одна лента	Пул соединений Prisma восстановился сам, потоки событий не обрывались

Итог прогона: 39 902 запроса, 538 ошибок (1,35 %) — все в окнах отказов; p95 чтения 30 мс, записи 51 мс. Потоки событий: 1072 обрыва и 1072 успешных переподключения (`hello` — 715 + 1072). Обновление токена ни разу не завершилось ошибкой, входить заново не пришлось никому.

Повторный прогон после исправления ошибки 8 (`results/failures`):

ОТКАЗ	ДО ИСПРАВЛЕНИЙ	ПОСЛЕ ИСПРАВЛЕНИЯ 8	ИТОГОВАЯ СБОРКА (8 И 9)
Аварийное завершение процесса API	0 ошибок, новый процесс за 1,3 с	0 ошибок, новый процесс за 0,9 с	0 ошибок, новый процесс за 0,9 с
Перезапуск API целиком	526 ошибок за 20 с; от `SIGTERM` до ответа нового API — 18 с	68 ошибок за 10 с; от `SIGTERM` до ответа — 3 с	**93 ошибки за 10 с; от `SIGTERM` до ответа — 3 с**
Перезапуск PostgreSQL	12 ответов `503`	13 ответов `503` и 2 ответа `500` — ошибка 9	**10 ответов `503`, ни одного `500`**; один из них — сырой запрос с кодом `57P01`
Итог	538 ошибок (1,35 %), 1072 сбоя потока событий	83 ошибки (0,21 %), 355 сбоев	**103 ошибки (0,26 %), 362 сбоя**

В итоговой сборке все ошибки — только в окнах отказов, в журнале сервера нет ни одной необработанной ошибки. Сбоев потока событий стало втрое меньше: при остановке API сервер закрывает потоки сам, и клиенты переподключаются без сбоя. Время перезапуска API — 3 с, из них около 1 с на запуск Nest. Разброс числа ошибок между прогонами (68–93) — запросы, пришедшие за эти секунды.

9 Рост данных (E51-08)

`growth.sh`: 2000 сотрудников с историей за пять прошлых лет. Замерены размеры таблиц до и после задачи сроков хранения, время копирования и восстановления.

ТАБЛИЦА	СТРОК ЗА 5 ЛЕТ	БЕЗ СРОКОВ ХРАНЕНИЯ	ПОСЛЕ ЗАДАЧИ `DATA-RETENTION`	ЧТО ХРАНИТСЯ
`email_outbox`	83 953	37 МБ	0	Письма — 90 дней
`refresh_tokens`	79 556	29 МБ	26 МБ	Сеансы — 30 дней, все строки наполнения — живые сеансы последнего месяца
`notifications`	83 953	15 МБ	0	Прочитанные уведомления — год
`login_events`	54 339	9,7 МБ	9,5 МБ	Журнал входов — по правилу организации, по умолчанию 365 дней; наполнение — один год
`audit_events`	50 214	9,3 МБ	9,1 МБ	Журнал изменений — не удаляется
`request_events`	50 214	5,1 МБ	5,1 МБ	История заявок — не удаляется
`vacation_requests`	35 318	4,9 МБ	4,7 МБ	Заявки — не удаляются
`vacation_notices`	23 744	3,8 МБ	3,7 МБ	Извещения — не удаляются
База целиком		**125 МБ**	**67 МБ**	

Выводы:

- **Документы кадрового учёта растут примерно на 4,5 МБ в год** на 2000 сотрудников: заявки, история заявок, журнал изменений и извещения — 22,6 МБ за 5 лет. Журнал входов и сеансы ограничены сроками хранения: около 10 и 26 МБ. За 10 лет база вырастет примерно до 100 МБ.
- **Письма и уведомления без сроков хранения росли бы на 10 МБ в год** — до 1.0.0 они не удалялись (недостаток F-10 самооценки ИБ). Наполнение — полные прошлые годы, поэтому задача удалила все письма и все уведомления истории. В работе в базе остаются письма за 90 дней и прочитанные уведомления за год — ещё около 5 МБ.
- **Копирование и восстановление занимают секунды:** `pg_dump -Fc` — 6,9 МБ за 1,2 с, `pg_restore -j 2` в пустую базу — 1,9 с. Копии за 14 дней — около 100 МБ.

Планы запросов на данных 5 лет. Полный просмотр большой таблицы — только у графика организации по `vacation_requests`. При 35 тыс. строк это 3 мс из 25 мс запроса (`EXPLAIN ANALYZE`); остальное — формирование 5 тыс. строк ответа, и ответ кэшируется по версии данных. За 7 часов `soak` таблица выросла до 54 тыс. строк, средний запрос — 28 мс. Журналы читаются только по индексам: за 5 часов прогона у `audit_events` и `request_events` — ни одного полного просмотра, у `refresh_tokens` — 3, у `notifications` — 36.

Журнал транзакций для архива WAL: 409 МБ за 7 ч 15 мин нагрузки часа пик, около 56 МБ в час. Верхняя оценка — 0,5 ГБ за рабочий день, при хранении 7 физических копий архив WAL — до 3,5 ГБ.

10 Воспроизведение

```
cd server && npm run build
# сайзинг: каталог результатов, численность, ядра стенда, профиль
GEN_CPUS=2,3 ../tools/load/pinned.sh /tmp/sizing/2000-1cpu-average 2000 0 average
../tools/load/pinned.sh /tmp/sizing/2000-1cpu-spike 2000 0 spike
../tools/load/pinned.sh /tmp/sizing/2000-1cpu-stress 2000 0 stress
../tools/load/pinned.sh /tmp/sizing/2000-1cpu-soak 2000 0 soak
# пики: почта — на SMTP-приёмник, на фоне average
node ../tools/load/smtp-sink.mjs --port 2525 --out mail.csv &
SMTP_HOST=127.0.0.1 SMTP_PORT=2525 ../tools/load/pinned.sh /tmp/peaks 2000 0 average --duration 1500 &
node ../tools/load/peaks.mjs --url http://127.0.0.1:8097 --db
postgresql://vacation:vacation@localhost:5432/vacation_load --approve
# отказы и рост данных
../tools/load/failures.sh /tmp/failures 2000
../tools/load/growth.sh /tmp/growth 2000 5
```